ATI-C01

Operating System

K. Reshma
192421256

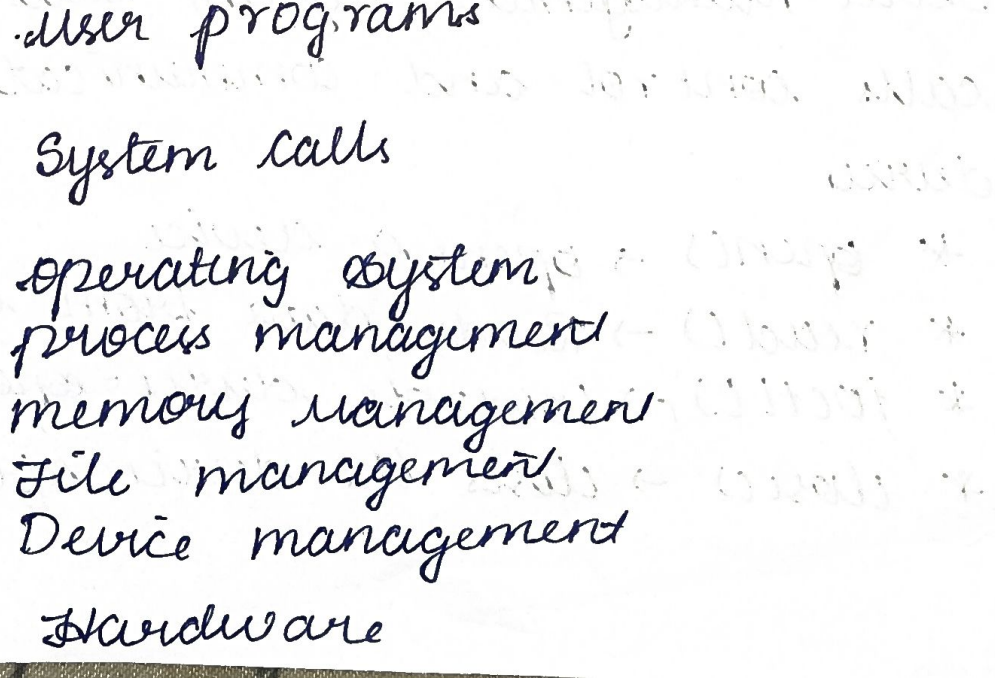
1. a) Types of operating System

1. multi-user OS - Allows many users to use the system at the same time.
2. multitasking OS - Runs multiple processes simultaneously.
3. Time-sharing OS - shares CPU time among users efficiently.

justification:

A multi-user Time-sharing Operating System is best because it allows multiple users and processes to share CPU, memory and I/O device efficiently

b) OS structure Diagram:



c) System call sequence

```
pid = fork();
if(pid == 0)
{
  exec("program");
}
else
{
  wait();
}
```

System calls: fork() -> exec() -> wait()

---

2) a) categorize system calls used for file and device management.

File Management system calls: These system calls are used to create, access, modify, and delete files.

* open() → opens a file
* read() → reads data from a file
* write() → write data to a file
* close() → close the file after use

Device Management system calls: These system calls control and communicate with hardware devices

* open() → opens a device
* read() → Reads data from a device
* ioctl() → controls device-specific operation
* close() → closes the device after use.

b) Illustrate how the OS handles file operations using a layered structure diagram.

Explanation: The operating system uses a layered structure to handle file operations. Each layer performs a specific function and communicates with the next layer.

User Application

File System

Device Drivers

Hardware

c) Write Pseudo-code or C-based system call sequence to open, read, and close a file

```
# include <fcntl.h>
# include <unistd.h>
int main()
{
  int fd;
  char buffer[100];
  fd = open("file.txt", O_RDONLY);
  if (fd != -1)
  {
    read(fd, buffer, 100);
    close(fd);
  }
  return 0;
}
```

3) a) Identify the type of OS and explain why it is suitable for real-time application

Type of operating system:

Real-Time operating system (RTOS)

Explanation:

A Real-Time operating system is designed to process data and respond to events within a fixed time limit. it is commonly used in embedded systems such as industrial machine robotics, medical equipment, and automotive control system. RTOS ensures fast response, high reliability, and accurate task execution without delays, making it suitable for real-time applications where is timing is critical.

b) Draw a diagram representing the interaction between hardware, kernel and user process.

- User Processes
(Application Tasks)

↓

Kernel

Process Management
memory Management
Device management

↓

Hardware

CPU, memory, I/O

() List and explain system calls required for process synchronization in this system

System calls for process synchronization:

* fork() - creates a new child process.
* wait() - makes the parent process wait until the child process finishes
* sem_wait() - Decreases the semaphore value and waits if the resource is unavailable.
* sem_post() - Increases the semaphore value and releases the waiting process

purpose:

These system calls help synchronize multiple processes, prevent resource conflicts and ensure that tasks are executed in the correct order in a real-time operating system.

Example sequence:

```
fork();
sem_wait();
sem_post();
wait();
```

4) a) classify the type of operating system used and compare it with a centralised OS.

Type of operating System:

Distributed operating System

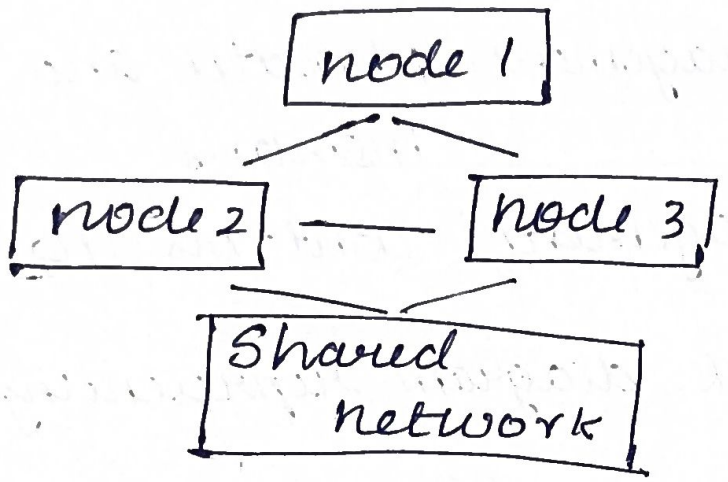
Explanation:

A Distributed OS manages multiple interconnected computers that work together as a single system. it allows us to share resource such as files, memory, printers and processing power over a network.

comparison:

Distributed OS	centralised OS
uses multiple interconnected computers	uses only one central computer
Resources are shared across all nodes.	Resources are available on the central system
high reliability and fault tolerance	Failure of the central affects all users.
Better Performance through load sharing	limited performance due to single system.

b) Draw the architecture of Distributed OS showing communication between nodes.



Explanation:

* Each Node is a independent Computer
* The network connects all node.
* nodes communicate and share files, memory and other resource network.

c) write the sequence of system calls involved in Inter-Process communication (IPC).

Example using pipe:

```
#include <unistd.h>
int fd[2];
char msg[] = "hello";
pipe(fd);
write(fd[1], msg, 6);
read(fd[0], msg, 6);
close(fd[0]);
close(fd[1]);
```

5) a) Mayor Function Of OS

1. Process scheduling - Allocate CPU to processes.
2. Memory management - Allocate and deallocate memory
3. Device management - controls I/O device

b) Draw a block diagram representing OS

User Application

↓

Operating System
process management
memory management
File management
Device management

↓

Hardware
CPU, memory, I/O

c) write system call examples for memory allocation and device management

memory Allocation:

```
#include <stdlib.h>
int *p;
P = (int *)malloc(10 * sizeof
        (int));
free (P);
```

Device Management:

```
#include <fcntl.h>
#include <unistd.h>
int fd;
fd = open ("device", O_RDWR);
read (fd, buffer, 100);
write (fd, buffer, 100);
close (fd);
```